

Operating Systems and Concurrency

Introduction 2

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2026

Recap

Last Lecture

- 1 The **operating systems** provides an abstraction of the actual hardware and **manages resources**
- 2 The **code** we write heavily **uses** operating system functionality

Goals for Today

Overview

- **Hardware** from an operating system's point of view
- **Address spaces**

Computer Hardware

Computer hardware in a nutshell

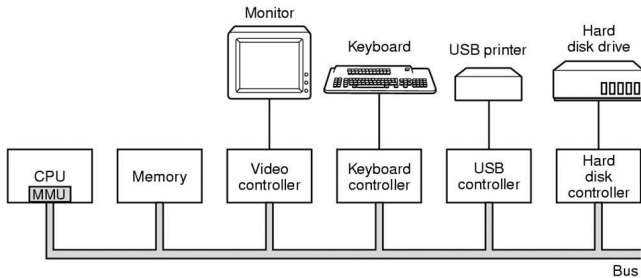


Figure: Simplified computer model (Tanenbaum, 2014)

Computer Hardware

CPU Design: Instruction evaluation

- Naively, CPU's execute a sequence of instructions one at a time
- A CPU's basic cycle consist of fetch, decode, and execute running in a pipeline
- A superscalar CPU provides instruction level parallelism, evaluating multiple instructions in parallel

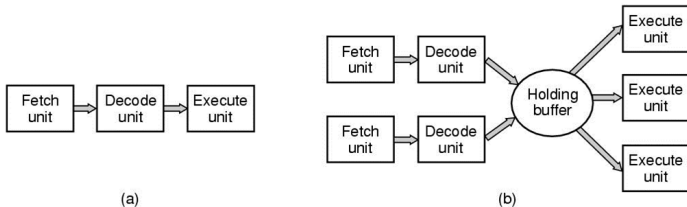


Figure: Processor Architectures (Tanenbaum, 2014) ¹

Computer Hardware

CPU Design: Advanced instruction evaluation

The search for further speed

- **Out-of-order execution** - instructions may not be evaluated in the order they appear in your code
- **Speculative evaluation** - speculative evaluation of instructions
- ... **predict**

Computer Hardware

CPU Design: Advanced instruction evaluation

The search for further speed

- **Out-of-order execution** - instructions may not be evaluated in the order they appear in your code
- **Speculative evaluation** - speculative evaluation of instructions
- ...

Take home message

You have to be very careful about the assumptions you make about CPU behaviour!^a

^aCompiler optimizations and memory architecture further complicate the picture

Computer Hardware

CPU Design: Registers

- Registers are small fast element of memory close to the CPU
- Registers have multiple purposes:
 - program counter (PC) - which instruction should be run next
 - program status word (PSW) - flags configuring the state of the CPU
 - general purpose registers - storing operands for CPU instructions
- The **compiler/programmer** decides what to keep in the registers
- **Registers** are part of the state of a running program

Computer Hardware

CPU Design: Moore's "law"

Moore's law

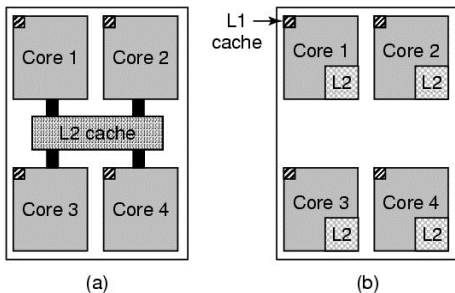
"The number of transistors on an integrated circuit (chip) doubles roughly every two years"

- Closely linked, but **not necessarily related to performance** (performance roughly doubles until 2003)
- Moore's still continuing, but the "**power wall**" slows performance improvements of single core/single processor systems
 - A few cores for multiple "programs" is easy to justify
 - How to use **massively parallel** computers/CPU's/many core machines?

Computer Hardware

CPU Design: Multiple cores and threads

- Modern CPUs contain **multiple cores** and are often support multiple hardware threads within a core
- **Evolution in hardware** has implications on **operating system design**
 - Older operating systems such as Windows XP did not support multi processor architectures
 - Modern operating systems, along with other infrastructure such as compilers, **need to make good use of hardware concurrency** to fully exploit hardware advances



Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

- If we run this multiple times, will the **same or different addresses** be displayed for `&iVar`?

Computer Hardware

CPU Design: Memory Management Unit

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

int main() {
    while(iVar++ < 10) {
        printf("Addr:%p; Val:%d\n", &iVar, iVar);
        sleep(1);
    }
}
```

- If we run this multiple times, will the **same or different addresses** be displayed for `&iVar`? **same**
- If we run two copies simultaneously, will the **same or different addresses** be displayed for `&iVar`? **usually the same (virtual address)**

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

Addr:0x10eac2000; Val:1 ...

Instance two output

...

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

Addr:0x10eac2000; Val:1 ...

Addr:0x10eac2000; Val:2 ...

Instance two output

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

Addr:0x10eac2000; Val:1

Addr:0x10eac2000; Val:2

Addr:0x10eac2000; Val:3

Instance two output

...

...

Addr:0x10eac2000; Val:1

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

Addr:0x10eac2000; Val:1

Addr:0x10eac2000; Val:2

Addr:0x10eac2000; Val:3

Addr:0x10eac2000; Val:4

Instance two output

...

...

Addr:0x10eac2000; Val:1

Addr:0x10eac2000; Val:2

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

Addr:0x10eac2000; Val:1
Addr:0x10eac2000; Val:2
Addr:0x10eac2000; Val:3
Addr:0x10eac2000; Val:4
...

Instance two output

...
...
Addr:0x10eac2000; Val:1
Addr:0x10eac2000; Val:2
...

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

```
Addr:0x10eac2000; Val:1  
Addr:0x10eac2000; Val:2  
Addr:0x10eac2000; Val:3  
Addr:0x10eac2000; Val:4  
...
```

Instance two output

```
...  
...  
Addr:0x10eac2000; Val:1  
Addr:0x10eac2000; Val:2  
...
```

- Both processes display 0x10eac2000, the same address.
- The value of `iVar` is different in each run of the program.

Computer Hardware

CPU Design: Memory Management Unit

Instance one output

```
Addr:0x10eac2000; Val:1  
Addr:0x10eac2000; Val:2  
Addr:0x10eac2000; Val:3  
Addr:0x10eac2000; Val:4  
...
```

Instance two output

```
...  
...  
Addr:0x10eac2000; Val:1  
Addr:0x10eac2000; Val:2  
...
```

- Both processes display 0x10eac2000, the same address.
- The value of `iVar` is different in each run of the program.

Different behaviour on Mac due to **Address Space Layout Randomization (ASLR)**.

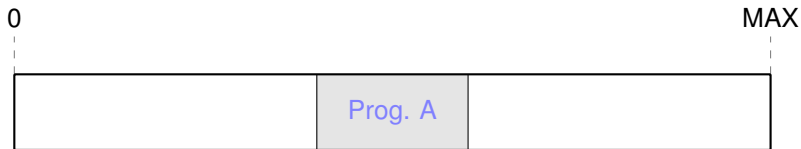
Computer Hardware

CPU Design: Memory Management Unit



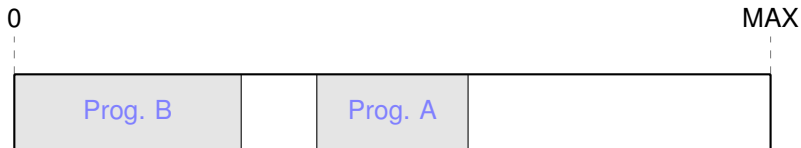
Computer Hardware

CPU Design: Memory Management Unit



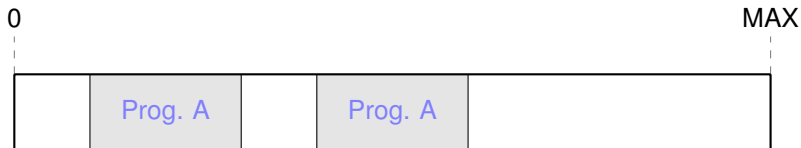
Computer Hardware

CPU Design: Memory Management Unit



Computer Hardware

CPU Design: Memory Management Unit



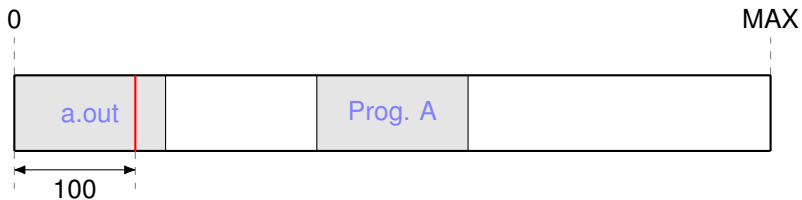
Computer Hardware

Memory Management Unit

- One cannot know **where in memory an executable will run**
 - Might be influenced by what other code is running on the machine
 - Multiple instances of the same program require distinct portions of memory
- When coding: variables have memory associated with them - they **need to have an address**
 - But the exact **physical addresses** cannot be known at compile time
 - The compiler just **assumes** that the program will **start at address 0**
 - If the program does not run at physical address 0 but at, e.g. 1000, just **add the offset (1000)** to every “compiled address”

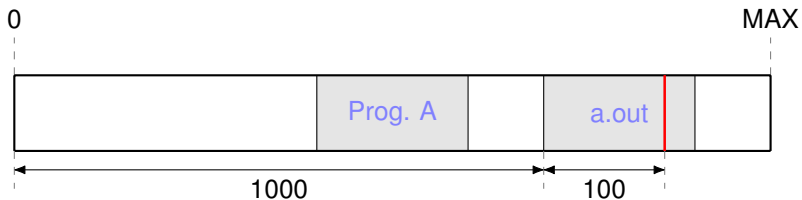
Computer Hardware

CPU Design: Memory Management Unit



Computer Hardware

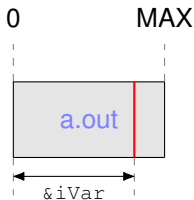
CPU Design: Memory Management Unit



Computer Hardware

Memory Management Unit

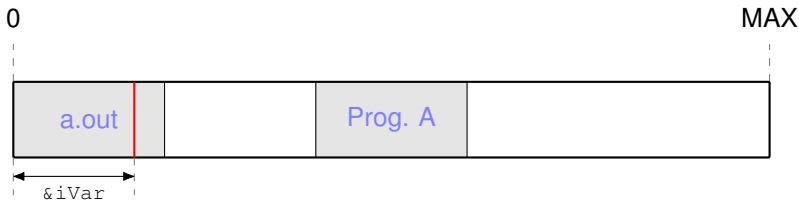
- There are two different address spaces:
 - the **logical address space** seen by the program and used by the compiler
 - the **physical address space** seen by the hardware/OS
- When compiling code, **memory addresses must be assigned to variables** and instructions, the compiler does not know what **memory addresses will be available** in physical memory
- It will just assume that the code will **start running at address 0** when generating the machine code



Computer Hardware

Memory Management Unit

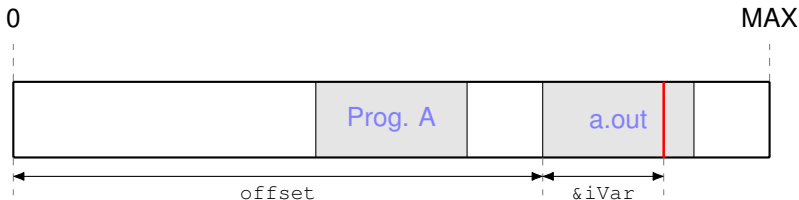
- There are two different address spaces:
 - the **logical address space** seen by the program and used by the compiler
 - the **physical address space** seen by the hardware/OS
- When compiling code, **memory addresses must be assigned to variables** and instructions, the compiler does not know what **memory addresses will be available** in physical memory
- It will just assume that the code will **start running at address 0** when generating the machine code



Computer Hardware

Memory Management Unit

- There are two different address spaces:
 - the **logical address space** seen by the program and used by the compiler
 - the **physical address space** seen by the hardware/OS
- When compiling code, **memory addresses must be assigned to variables** and instructions, the compiler does not know what **memory addresses will be available** in physical memory
- It will just assume that the code will **start running at address 0** when generating the machine code



Computer Hardware

Memory Management Unit

- On some rare occasions, the program **may run at physical address 0**
 - `physical address = logical address + 0`
- On other occasions, it will be running at a **completely different location in physical memory and an offset is added**
 - `physical address = logical address + offset` *&ivar*
- The **memory management unit (MMU)** is responsible for **address translation** (“adding the offset”)
 - Different program instances require **different address translation**
 - The state of the MMU is **part of the state of a running program**

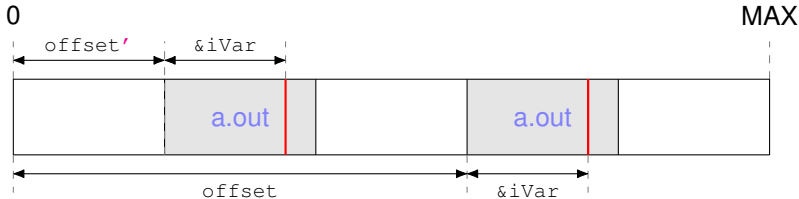
Computer Hardware

CPU Design: Memory Management Unit

- In the case of our example:
 - The address printed on the screen is the **logical address**
 - The logical address is translated into **two different physical addresses** using **different offsets**
 - The code

```
printf("Addr:%p; Val:%d\n", &iVar, iVar);
```

was printing the logical address of iVar.



Computer Hardware

Memory Management Unit

- Our discussion has omitted many details, address translation may be more complex than applying a single offset uniformly. More detail in memory management lectures later.
- The key point is that the data required by the MMU to translate addresses is part of the state of a running program

Recap

Take-Home Message

- Operating Systems design is heavily coupled to hardware design and capabilities
- Registers contents and data needed by the MMU for address translation are **part of the state of a running program**
- The MMU abstracts the specifics of the actual memory hardware

Test your understanding

- 1 Can a user program choose to write to a specific location in *physical memory*?
Nope Usually no
- 2 Can a user program find out where a variable lives in *physical memory*?
- 3 Are the answers to these questions a good thing or a bad thing?
Yes: Memory protection, Security, Simplifies programming

Introduction 2 – Core Revision Summary

Hardware from the OS viewpoint · CPU execution · registers · hardware concurrency · MMU · logical vs physical address spaces

◆ source-marked / annotated content

1. Topic Overview

This lecture links **hardware behaviour** to **operating-system design**. It starts with how CPUs execute instructions and preserve execution state, then explains why modern multicore hardware requires OS-level concurrency support. The main thread is **memory abstraction**: a process works with a logical address space, while the OS/MMU maps that view onto physical memory.

Lecture logic: CPU execution model → registers / running-program state → multicore hardware → memory placement problem → logical vs physical addresses → MMU translation and protection.

2. Core Concepts

Operating system and hardware

- ▶ ◆ The OS provides an **abstraction of the actual hardware** and **manages resources**; normal application code heavily depends on OS functionality.
- ▶ Simplified machine view (p.4): CPU + MMU, memory, and I/O controllers communicate over a bus. OS design is therefore tightly coupled to hardware capabilities.

CPU instruction execution

- ▶ ◆ Basic CPU cycle: **fetch** → **decode** → **execute**; pipelining overlaps these stages.
- ▶ ◆ A **superscalar CPU** exploits **instruction-level parallelism**: multiple instructions can be evaluated in parallel.
- ▶ ◆ **Out-of-order execution**: evaluation order may differ from source-code order. **Speculative evaluation**: the CPU may predict and execute work before it is known to be needed.
- ▶ ◆ **Take-home**: do not make simplistic assumptions about CPU behaviour; compiler optimisations and memory architecture complicate the observable execution model further.

Registers and running-program state

- ▶ Registers are small, fast memory elements close to the CPU. Their contents are part of the **state of a running program**.
- ▶ ◆ **Program Counter (PC)**: identifies the instruction to run next.
- ▶ ◆ **Program Status Word (PSW)**: flags / control bits describing CPU state.
- ▶ ◆ **General-purpose registers**: hold operands and intermediate values; compiler/programmer decides what is kept in them.

Moore's law, the power wall, and hardware concurrency

- ▶ Moore's law: transistor count on an integrated circuit roughly doubles every two years; transistor growth is **not identical to performance growth**.
- ▶ Single-core performance scaling slowed at the **power wall**; modern CPUs therefore use multiple cores and often multiple hardware threads per core.
- ▶ ◆ Hardware evolution affects OS design: modern OSs and compilers must exploit **hardware concurrency** to benefit from multicore systems.

Address spaces and the MMU

- ▶ An executable cannot know in advance exactly **where in physical memory** it will run; placement depends on the rest of the running system.
- ▶ Multiple instances of the same program need **distinct physical memory regions**, even if their program-visible addresses look the same.
- ▶ ◆ **Logical address space**: addresses seen by the program and used by the compiler. **Physical address space**: addresses used by hardware/OS.
- ▶ ◆ The **Memory Management Unit (MMU)** performs address translation. The translation data held by the MMU is part of the **state of a running program**.

3. Key Mechanisms / Algorithms

1) CPU execution path

Instruction stream → Fetch → Decode → Execute

Pipelining overlaps stages; superscalar execution adds parallel functional paths; out-of-order/speculative techniques may change the timing/order of evaluation.

2) Why address translation is needed

1. Program variables/instructions must have addresses in generated machine code.
2. At compile time, the compiler does **not** know what physical memory will be free when the program runs.
3. ♦ The lecture's simplified model assumes code is compiled relative to logical address 0.
4. At runtime, the process may be placed elsewhere in physical memory, so an address translation is applied.
5. Different process instances use different translation state, keeping their physical memory separate.

3) Worked example: two instances of the same program

For the source example with global variable `iVar`, two simultaneous processes can print the **same program-visible address** but hold **different values**.

- ▶ ♦ Annotated answer (p.18): repeated run – address can be the **same**; simultaneous copies – **usually the same virtual/logical address**.
- ▶ ♦ Observed in the example (pp.19–25): both instances display `0x10eac2000`, while each process has its own `iVar` value.
- ▶ The same logical address is translated into **two different physical addresses** using different process-specific translation state.
- ▶ Source note: behaviour can differ on macOS because of **Address Space Layout Randomization (ASLR)**.

4) MMU translation path

Program uses logical address → MMU applies process-specific translation → physical memory address

The lecture illustrates translation as adding an offset, but explicitly notes that real address translation can be more complex.

4. Formula / Rules / Conditions

- ▶ Simplified lecture rule:

$$\text{physical address} = \text{logical address} + \text{offset}$$

- ▶ Special case: if loaded at physical address 0, then $\text{offset} = 0$ and $\text{logical} = \text{physical}$ in this simplified model.
- ▶ ♦ Different program instances require **different address translation**; therefore their MMU translation state differs.
- ▶ The address printed by `printf("%p", &iVar)` in the lecture example is the **logical address**, not a guaranteed physical-memory location.
- ▶ ♦ User programs generally cannot choose an arbitrary physical memory location to write to and usually cannot determine a variable's physical address directly.
- ▶ These restrictions are desirable because they enable **memory protection**, improve **security**, and **simplify programming** (annotated answers, p.40).

Conditions / limits to remember

- ▶ The single-offset model is a **teaching simplification**; the lecture defers real memory-management details to later lectures.
- ▶ Same logical address $\neq$ same physical address across processes.
- ▶ Same executable $\neq$ shared mutable variable state by default: separate process instances need separate physical memory.
- ▶ ASLR can change the address values observed between runs; do not assume identical virtual/logical placement on every platform/run.

5. Comparison & Common Confusions

Concept A	Concept B	Key Difference / Revision Point
Sequential CPU view	Pipelined / superscalar CPU	Sequential is a simple model; pipelining overlaps stages, superscalar evaluates multiple instructions in parallel.
In-order execution	Out-of-order / speculative execution	Hardware may evaluate work in a different order or predict future work; source-code order alone is not a safe timing assumption.
Program Counter (PC)	Program Status Word (PSW)	PC says <i>what instruction comes next</i> ; PSW stores CPU status/configuration flags.
Logical address	Physical address	Logical: process/compiler-visible. Physical: hardware/OS-visible memory location.
Same logical address	Same physical address	They are not equivalent. Two processes may show the same logical address while MMU translation maps them to different physical locations.
Compiler-time address	Runtime placement	Compiler assigns program-relative/logical addresses without knowing which physical region will be available at runtime.
MMU translation state	CPU register state	Both contribute to the state of a running program: registers capture execution state; MMU state captures address translation.
More transistors	More single-core performance	Moore's law concerns transistor count; performance is related but not guaranteed to scale identically, especially after the power wall.

6. Exam / Revision Focus

Must Know

- ▶ Definitions: **logical address space**, **physical address space**, **MMU**, **PC**, **PSW**, **superscalar**, **out-of-order execution**, **speculative evaluation**, **ASLR**.
- ▶ CPU basic cycle: **fetch** → **decode** → **execute**; superscalar = instruction-level parallelism.
- ▶ Lecture formula/model: $\text{physical} = \text{logical} + \text{offset}$; each process instance may use different translation state.
- ▶ Registers and MMU translation data are both part of the **state of a running program**.

Must Understand

- ▶ Why a compiler cannot bind ordinary program variables to final physical addresses at compile time.
- ▶ How two processes can print the same logical address yet access different physical memory and therefore hold different values.
- ▶ Why multicore/hardware-thread evolution forces OSs and compilers to support concurrency effectively.
- ▶ Why hiding physical memory from user programs improves isolation, protection, security, and programming simplicity.

Easy to Confuse

- ▶ **Printed pointer address in this lecture example** = logical/virtual address, not necessarily a directly observable physical address.
- ▶ **Same address shown by two processes** does not mean shared memory.
- ▶ **Moore's law** is about transistor count, not a law that CPU performance must double every two years.
- ▶ **Offset translation** is the lecture's simplified explanatory model; real MMU translation is more complex.
- ▶ Do not assume instructions execute strictly in source-code order because of pipelining, superscalar, out-of-order, speculative execution, compiler optimisation, and memory architecture.